04

Python cho Data Engineer

Pandas · Polars · DuckDB · xử lý file lớn

Hôm nay mình nói gì

1. Python của DE cần gì
2. Pandas vs Polars vs DuckDB
3. Đọc CSV / Parquet / JSON
4. Xử lý file lớn hơn RAM
5. Code sạch: type hints, logging

Python của DE không phải Python của DS

- Chạy không người canh
- Phải báo lỗi rõ
- Phải chạy lại được
- Phải đo được thời gian và bộ nhớ

Pandas vs Polars vs DuckDB

	Pandas	Polars	DuckDB
Mô hình	Eager, 1 lõi	Lazy, đa lõi	SQL engine
Bộ nhớ	Copy nhiều	Cột, ít copy	Streaming
Hợp với	Notebook nhỏ	Pipeline vừa	Query file trực tiếp
Cú pháp	DataFrame quen thuộc	Expression, chain	SQL thuần
Khi nào chọn	Thăm dò, < 1 GB	Transform trong code	Aggregate trên Parquet

Cùng một file Parquet, 3 cách đọc

```
# Pandas — eager
import pandas as pd
df = pd.read_parquet("pos/2026-10.parquet")

# Polars — lazy
import polars as pl
df = pl.scan_parquet("pos/2026-10.parquet").collect()

# DuckDB — SQL
import duckdb
df = duckdb.read_parquet("pos/2026-10.parquet").df()
```

scan = lazy, collect = chạy.

DuckDB query thẳng thư mục

```
import duckdb

result = duckdb.sql("""
    SELECT store_id, SUM(amount) AS revenue
    FROM 'pos/*.parquet'
    GROUP BY 1
    ORDER BY 2 DESC
""")

result.show()
```

Query thẳng thư mục Parquet, không cần load.

File lớn hơn RAM



Không bao giờ giữ cả file trong tay.

Đọc theo chunk, ghi Parquet theo ngày

```
import pandas as pd

reader = pd.read_csv("pos.csv", chunksize=500_000)

for i, chunk in enumerate(reader):
    chunk["day"] = pd.to_datetime(chunk["ts"]).dt.date
    chunk.to_parquet(
        "pos/",
        partition_cols=["day"],
        engine="pyarrow",
    )
    del chunk
```

500k dòng mỗi vòng; RAM đỉnh không phụ thuộc kích thước file.

Hàm pipeline: type hints + logging

```python
import logging
from datetime import date
import polars as pl

log = logging.getLogger(__name__)

def load_pos(day: date) -> pl.DataFrame:
    """Đọc giao dịch POS của một ngày từ Parquet."""
    df = pl.read_parquet(f"pos/day={day}/*.parquet")
    n = df.height
    log.info("loaded %s rows for %s", n, day)
    return df
```

Log có số, không log "done".

Cấu trúc project pipeline tối thiểu

src/	Code pipeline
tests/	pytest
pyproject.toml	Dependencies + ruff
.env.example	Biến môi trường mẫu, không secret
Makefile	make run · make test

Đo trước khi tối ưu

- `time.perf_counter()` quanh từng bước
- `memory_profiler` cho hàm nghi ngờ
- So sánh trên cùng dữ liệu
- Ghi kết quả vào README lab

Lỗi hay gặp

SAI	ĐÚNG
`for row in df.iterrows()`	Vector hoá hoặc Polars
~~Đọc CSV mỗi lần chạy~~	Chuyển sang Parquet một lần
`print` ~~để debug~~	logging có level
~~Nuốt exception bằng~~ `except: pass`	Log và raise

Lab #04 — Chuyển 2 GB CSV sang Parquet

1. Sinh 2 GB CSV: `coffeegen pos --days 365`
2. Chuyển bằng Pandas chunk, Polars, DuckDB
3. Đo thời gian và RAM đỉnh từng cách
4. Query SUM theo store: Parquet vs CSV

KẾT QUẢ MONG ĐỢI

Parquet nhỏ hơn **5–8 lần**

Query nhanh hơn **≥ 10 lần**

CÂU HỎI PHỎNG VẤN #04

Vì sao Parquet nhanh hơn CSV?

Columnar, nén theo cột, có metadata min/max để bỏ qua row group.

VIDEO TIẾP THEO

05

Docker cho Data Engineer: dựng Postgres, Airflow, Spark trong 20 phút

Thứ Ba 17/11 · 19:00

Đăng ký